

洛谷

深入浅出 程序设计竞赛

(基础篇)

洛谷学术组 编著
汪楚奇



高等教育出版社·北京

内容提要

本书分为4部分:第1部分介绍C++语言的基础知识,包括表达式、变量、分支、循环、数组、函数、字符串、结构体等内容;第2部分介绍一些基础算法,包括模拟、高精度、排序、枚举、递推、递归、贪心、二分、搜索等;第3部分介绍几种简单常用的数据结构,包括线性表、二叉树、并查集、哈希表和图;第4部分是在算法竞赛中需要使用的数学基础,包括位运算与进制转换、计数原理、排列与组合、质数与合数、约数与倍数等概念。

本书主要面向从未接触过程序设计竞赛(包括NOI系列比赛、ICPC系列比赛)的选手,也适用于稍有接触算法、希望进一步巩固算法基础的读者。

本书提供一些在线的配套资源,例如课件或勘误表,读者可以发邮件至编辑邮箱1548103297@qq.com索取。

图书在版编目(CIP)数据

深入浅出程序设计竞赛.基础篇 / 汪楚奇编著. —
北京:高等教育出版社,2020.10(2022.5重印)
ISBN 978-7-04-054276-9

I. ①深… II. ①汪… III. ①程序设计-教材 IV.
①TP311.1

中国版本图书馆CIP数据核字(2020)第102486号

策划编辑 侯昀佳
插图绘制 邓超

责任编辑 刘子峰
责任校对 李大鹏

封面设计 赵阳
责任印制 高峰

版式设计 杜微言

出版发行 高等教育出版社
社址 北京市西城区德外大街4号
邮政编码 100120
印刷 人卫印务(北京)有限公司
开本 787 mm×1092 mm 1/16
印张 20.5
字数 460千字
购书热线 010-58581118
咨询电话 400-810-0598

网 址 <http://www.hep.edu.cn>
<http://www.hep.com.cn>
网上订购 <http://www.hepmall.com.cn>
<http://www.hepmall.com>
<http://www.hepmall.cn>
版 次 2020年10月第1版
印 次 2022年5月第5次印刷
定 价 59.50元

本书如有缺页、倒页、脱页等质量问题,请到所购图书销售部门联系调换
版权所有 侵权必究
物料号 54276-00

前言

在讲到本书之前,我想先介绍一下令我引以为傲的洛谷。

我在 2013 年被保送到上海交通大学后,为母校厦门一中开发了洛谷 Online Judge 并向公众公开。经过长年累月的持续开发运营,洛谷已经成为了一个拥有数十万用户的知名程序设计竞赛题库和社区。我因为洛谷能够帮助这么多用户提升编程水平而感到非常的荣幸。

随着信息技术的不断发展,越来越多的同学开始学习程序设计和算法。他们学习算法可能是为了参加 NOI 系列比赛、ICPC 系列比赛,也可能是为了求职、升学,甚至只是单纯因为喜好,享受算法之美。

但是光有题库还是不够的,很多同学希望能够有人指导他们如何学习。因此,洛谷举办了几次在线的算法竞赛课程,受到的评价颇佳,同时积累了不少宝贵的教学资源。为了让更多的人能够接触到这些资源,我们对这些资源进行了整合并加上了一些补充,于是深入浅出程序设计竞赛系列书问世了。

本书特色

本书是深入浅出程序设计竞赛系列书的第一本,主要面向从未接触过程序设计竞赛(包括 NOI 系列比赛、ICPC 系列比赛)的选手,也适用于稍有接触算法、希望进一步巩固算法基础的读者。本书不同于其他同类书籍,具有以下几个主要特点。

特点 1:覆盖重要的初级知识点,打好坚实的基础。

本书从介绍 C++ 语言开始,各个击破程序设计竞赛中的基础考点,包括各种基础算法、数据结构和数学知识。这些都是重中之重,如果没有掌握这些知识,继续深入的学习就如同试图建设空中楼阁。这些知识对读者的数学水平要求不高,有初中的数学基础即可阅读。

特点 2:深入浅出,解答“是什么、为什么、怎么办”的问题。

正如本书的书名一样,本书力求使用浅显易懂的语言讲述各种深刻的算法思想,因此读起来生动有趣,而不是面对冰冷的数学符号和代码。每个专题中都会以精心选择的例题为主线,

先介绍这个知识点是用来干什么的;然后花费大量的篇幅,图文并茂地介绍各个知识点的详细过程和代码的实现方法;最后结合题目,介绍如何将算法应用到实战中。有些例题甚至介绍了多种不同做法,帮助读者拓展思维,举一反三。

特点 3:充满干货的经验之谈。

本书的多数例题都配备代码风格良好的示例代码,可以帮助读者更好地学习如何使用代码实现算法。在讲解的过程中给出了大量的算法竞赛中需要注意的琐碎问题,这些都是前人的经验之谈,希望读者可以少走弯路。

特点 4:讲练结合,提供大量的练习机会。

本书有约 400 道例题和习题。为了节约阅读时间和篇幅,例题和习题都只保留了题意简述。大部分的题目都可以在洛谷中找到并提交代码评测,还可在洛谷阅读其他同学提供的题解并进行讨论。题目选材范围多样,覆盖了应当学习了解的知识点。如果读者能够完全掌握这些题目,相信可以有相当程度的进步。

内容安排

本书作为系列书的第一本,分为 4 部分。

第 1 部分介绍 C++ 语言的基本知识,包括表达式、变量、分支、循环、数组、函数、字符串、结构体等内容,后续所有内容都需要使用到这些语言基础。同时,辅以一些需要思考的入门算法题目,使读者在学习语言的过程中初步接触到算法的思维之美。

第 2 部分介绍一些基础算法,包括模拟、高精度、排序、枚举、递推、递归、贪心、二分、搜索等。这些基础算法是程序设计竞赛中的重要部分,读者学习完这一部分可以解决一些简单算法类题目。

第 3 部分介绍几种简单常用的数据结构,包括线性表、二叉树、并查集、哈希表和图。数据结构是存储和操作数据的骨架,许多算法的实现需要依赖各种数据结构。

第 4 部分是在算法竞赛中需要使用的数学基础,包括位运算与进制转换、计数原理、排列与组合、质数与合数、约数与倍数等概念。数学问题也是算法竞赛的常考点,虽然相对比较抽象,但也应当掌握。

最后在附录中提供了程序设计环境配置,以及算法评价与复杂度的相关内容。这些内容不太适合放在前面任何章节中,但却是非常重要的参考资料,供需要的读者阅读。

学习建议

如果希望从新手成长为高手,学习的过程绝不是轻松的。学习如同登山,虽然希望通过本书和洛谷能助读者一臂之力,但仍要付出尽可能多的努力才行。希望读者可以做到以下几点。

1) 熟悉固定套路与算法模板:这些知识点学习起来很明确。本书已经整理出什么样的算法需要学习,读者应当理解每一种算法能干什么、适用于什么场合、算法复杂度是什么。除此之

外,还要反复地敲算法模板,从空白文件开始敲,直到一遍通过为止,这一点你必须很熟练。

2) 分析方法与编程经验:除了本书所涉及的数百道例题和习题,洛谷和其他题库也提供了许多非常丰富的题目资源,可以帮助读者查缺补漏。如果希望能够熟练掌握算法,增强思维敏捷性,在学习完算法与数据结构后,必须要大量地完成相关的题目。建议本书的读者花费半年时间学习完这本书,至少完成 300 题,且每周不少于 10 题。书中给出的代码仅仅是用来参考的,千万不能照抄。只有亲自动手实践,学到的东西才是自己的。

3) 保证程序正确:培养一次写对的能力,提交评测之前要谨慎。学会怎样调试自己的程序。就算感觉对拍办不到,也应该人工生成多组数据,手算结果,然后测试自己的程序对不对。所有犯过的错误都要写笔记,并且不再犯第二次。

对于没有任何编程基础的读者,建议从头开始按章节顺序学习。如果有一定的基础,那么可以选择性地阅读其中希望进一步巩固的部分。除了第 1 部分外,后面部分的各章相对独立,读者可以根据实际情况自行安排学习顺序。

大量写题是很重要的。特别是对于参加 NOI 等赛事的选手而言,不应拘泥于一本书或者一个题库,而是应根据自己的实际需求,广泛涉猎自己需要的资源。比较常见的编程在线题库(Online Judge)包括以下几个。

洛谷(www.luogu.com.cn):始于 2013 年,拥有强大的功能、活跃的社区、丰富且高质量的题目,是国内最为著名的 Online Judge 之一。本书例题和习题大多可以在这里找到。

LibreOJ(loj.ac):以自由著称,提供许多难度较高的题目,可以下载测试数据和查看其他用户代码。

POJ(poj.org)/HDU(acm.hdu.edu.cn):分别是由北京大学和杭州电子科技大学主办的在线题库,在 ACM 选手中较为知名,也有许多好题。

Codeforces(codeforces.com):俄罗斯题库,比较频繁地举办各种高质量在线比赛。

致谢

首先需要感谢厦门一中的吴旭日和赵艳老师,他们作为启蒙者将笔者带入了算法竞赛圈。

感谢当时和笔者一起创造洛谷的林梓楠同学、现在的洛谷科技合伙人林方正(lin_toto),以及金少海(soha)、施展(Darkflames)等开发者和各位为洛谷无私奉献的管理员们,还有洛谷网校的讲师们。没有你们就没有现在的洛谷。

参与本书编写的还有北京大学的周永隆(Flierking)、清华大学的李欣隆(nzhtl1477)、上海交通大学的徐牧辰(Scarlet)和哈尔滨工业大学的吴雨伦(ruanxingzhi)。尽管他们的平均年龄不到 20 岁,但都是在各项程序设计竞赛中取得斐然成绩的优秀选手。他们为本书提供了很多优质的内容。

在本书的编写过程中,参考了很多互联网和纸质资料,无法一一列举,在此表示诚挚的谢意。本书涉及的习题来源广泛,笔者已经尽可能在题目名称旁标注了题目来源或作者,但有些

题目无法确定来源,希望出题者能够联系我们认领这些题目,再版时会补充这些信息。

感谢笔者的朋友黄鸿排版了本书的试读稿,以及许多找到疏漏和提出改进意见的洛谷网校学员和一线教练朋友们。

最后感谢高等教育出版社的编辑,帮助我们顺利出版此书,也感谢所有阅读本书的朋友。

尽管为编写这本书花费了笔者很多心血,但因为时间仓促,难免会存在瑕疵与疏漏,欢迎读者朋友们指正或提出建设性意见,以便再版时改进。如果在阅读中发现任何问题,欢迎在洛谷的讨论版中发帖并 @kkksc03。

作 者

2020年6月

于上海

目 录

第1部分 语言入门.....001

第1章 简简单单写程序.....002

- 1.1 程序设计的目标和流程.....002
- 1.2 简单数学运算.....006
- 1.3 变量与常量.....009
- 1.4 课后习题与实验.....012

第2章 顺序结构程序设计.....014

- 2.1 变量的数据类型.....015
- 2.2 变量的输入与输出.....020
- 2.3 顺序结构程序设计案例.....024
- 2.4 提交评测与错误自查.....028
- 2.5 课后习题与实验.....030

第3章 分支结构程序设计.....032

- 3.1 关系表达式与逻辑表达式.....033
- 3.2 分支语句.....036
- 3.3 分支嵌套.....040
- 3.4 分支程序设计案例.....043
- 3.5 课后习题与实验.....045

第4章 循环结构程序设计.....047

- 4.1 for 语句和 while 语句.....048
- 4.2 多重循环.....053
- 4.3 循环结构程序设计案例.....056
- 4.4 课后习题与实验.....064

第5章 数组与数据批量存储.....067

- 5.1 一维数组.....067
- 5.2 多维数组.....071
- 5.3 数组应用案例.....074
- 5.4 课后习题与实验.....078

第6章 字符串与文件操作.....081

- 6.1 字符数组.....082
- 6.2 string 类型字符串.....087
- 6.3 文件操作与重定向.....091
- 6.4 课后习题与实验.....095

第7章 函数与结构体.....098

- 7.1 定义子程序.....099
- 7.2 变量作用域与参数传递.....102
- 7.3 递归函数.....105
- 7.4 结构体的使用.....108
- 7.5 课后习题与实验.....112

第2部分 初涉算法.....115

第8章 模拟与高精度.....116

- 8.1 模拟方法问题实例.....117
- 8.2 高精度运算.....121
- 8.3 课后习题与实验.....125

第9章 排序.....129

- 9.1 计数排序.....129
- 9.2 选择排序、冒泡排序、插入排序.....131
- 9.3 快速排序.....134
- 9.4 排序算法的应用.....136
- 9.5 课后习题与实验.....139

第10章 暴力枚举.....141

- 10.1 循环枚举.....142
- 10.2 子集枚举.....147
- 10.3 排列枚举.....149
- 10.4 课后习题与实验.....151

第11章 递推与递归.....153

11.1 递推思想	154
11.2 递归思想	158
11.3 课后习题与实验	162
第 12 章 贪心	164
12.1 贪心与证明	164
12.2 哈夫曼编码	169
12.3 课后习题与实验	172
第 13 章 二分查找与二分答案	174
13.1 二分查找	174
13.2 二分答案	179
13.3 课后习题与实验	184
第 14 章 搜索	186
14.1 深度优先搜索与回溯法	186
14.2 广度优先搜索	193
14.3 课后习题与实验	199

第 3 部分 简单数据结构

第 15 章 线性表	204
15.1 数组	205
15.2 栈	207
15.3 队列	211
15.4 链表	214
15.5 课后习题与实验	220
第 16 章 二叉树	223
16.1 二叉树的概念和建立	224
16.2 二叉树的遍历	228
16.3 二叉树的综合应用	230
16.4 课后习题与实验	235
第 17 章 集合	238
17.1 并查集	238

17.2 Hash 表	242
17.3 集合应用实例	246
17.4 课后习题与实验	250
第 18 章 图的基本应用	252
18.1 图的概念和建立	252
18.2 图的遍历	256
18.3 DAG 与拓扑排序	260
18.4 课后习题与实验	263

第 4 部分 基础数学与数论

第 19 章 位运算与进制转换	268
19.1 各种进制	269
19.2 二进制的深入探究	272
19.3 逻辑命题与位运算	274
19.4 课后实验与习题	277
第 20 章 计数原理与排列组合	279
20.1 加法原理和乘法原理	279
20.2 排列与组合	282
20.3 课后习题与实验	286
第 21 章 整除理论	288
21.1 整除的基本知识	289
21.2 质数与合数	291
21.3 最大公约数与最小公倍数	294
21.4 (选读)算术基本定理	298
21.5 课后习题与实验	301

附录

附录 A 程序设计环境配置	304
附录 B 算法评价与复杂度	311

An abstract graphic design featuring various shades of green. It includes a large white triangle in the upper right, a horizontal band of lighter green in the middle, and a darker green band at the bottom. The left side of the design is irregular and jagged, resembling a torn edge.

第 1 部分

语言入门

第1章 简简单单写程序

千里之行,始于足下。程序设计虽然花样繁多,但还是要从最简单的地方开始学习,由浅入深,直至掌握。毕竟,任何复杂的工程代码都是由一行行简单的代码组成的。

本章将讲解程序设计的基本概念,介绍程序的输入输出,并使用简单的语句完成一些简单的任务。(图 1-1 所示为本章思维导图。)

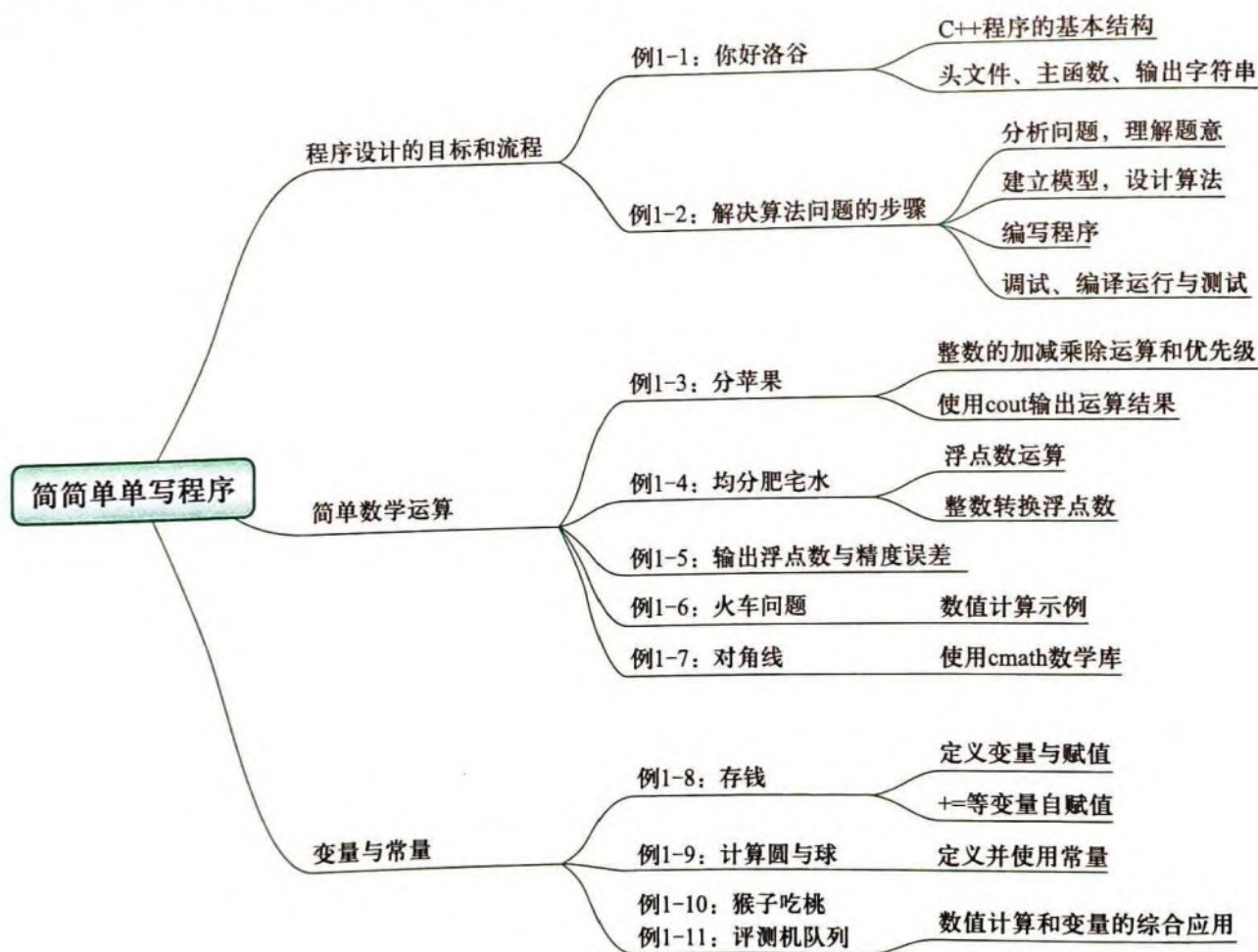


图 1-1 本章思维导图

1.1 程序设计的目标和流程

促使大家学习程序设计的原因可能有很多种:可能是为了参加各项算法竞赛,也可能是为

了协助处理数据,还有可能是编写游戏使人娱乐。但是归根结底,设计一个程序是为了让计算机始终不渝地遵循指令,完成特定的任务。为了能让计算机听懂指令,人们编写程序来与计算机交流。

正如人类有很多种语言一样,人和计算机交流也有很多种计算机语言。在工程中常见的语言有 C、C++、Java、Python、PHP、JavaScript 等。这些语言有不同的特性和适用范围,活跃在不同的场合,所以讨论“什么是最好的语言”是毫无意义的。很多工程师能掌握多门语言以应对不同的场合。

对于算法竞赛来说,NOI 系列比赛主要使用 C++^①,而 ICPC 系列比赛除了 C++,还能使用 Java、Python 等^②。本书使用 C++ 作为教学语言。几乎所有算法竞赛都允许使用 C++ 语言提交。

讲太多理论是非常枯燥的,因此直接开始上机实操帮助读者理解编程。即使现在不懂得这些理论也不影响学习编程。

“知其然而不知其所以然”并非是不值得提倡的。很多概念晦涩难懂,因此在现阶段的学习过程中直接照做而不用理解原理以节约时间和精力。在以后的学习过程中还有机会去深究这些概念的本质。

例 1-1 打开 IDE,输入下面的程序,并且编译运行。

如果没能看懂前面一句话是什么意思,也可以翻到附录 A 跟着做。

```
#include<iostream>
using namespace std;
int main() {
    cout << "I love Luogu!";
    return 0;
}
```

可以看到计算机的输出:

```
I love Luogu!
```

请注意,编写程序时必须和例子完全一致,除了双引号里面的内容。同时要注意所有的符号均为半角而不是全角^③,特别注意不要遗漏行末的分号,否则可能会出现编译错误。如果尝试运行的时候发现编译错误,可以阅读一下编译报错信息(不要怕读英语),会指出你在哪里犯了什么错误,修正好就可以了。

这个程序的目的是很简单,就是输出一句话。也可以改变双引号里面的内容,输出其他的任

① 早期允许使用 BASIC,2022 年前还允许使用 C 语言和 Pascal,但是使用比例已经很少。

② ICPC 2019 还允许使用 Kotlin 这种目前比较小众的语言,很有可能是因为该语言的发明公司 JetBrains 是 ICPC 的赞助商。

③ 建议编写程序时不使用中文输入法而使用英语输入模式。

何想输出的内容。因为编码的问题,现阶段不建议在程序的任何地方出现中文,包括程序的文件名和程序片段。如果没有处理好,则可能会出现乱码或者其他的问题。

下面来逐句分解这个程序。

`#include<iostream>` 是头文件,用于引入外部库。举个例子,想要烹饪一道菜,除了食材之外还需要一些其他工具,如锅和铲子。这次使用的 `iostream` 头文件是一个可以用来输入输出的“工具”,之后会接触更多的头文件,甚至可以自行编写头文件,也就是造一个工具。

`using namespace std;` 是命名空间声明语句。暂时不用理解它是干什么用的,但是现在使用到了 C++ 的一些语句(例如 `cin`),则应当加上这句话。注意,最后的分号不要遗漏。

`int main()` 是主函数。可以看到主函数后面有一对大括号 `{ }`,将主函数中的语句给包围起来了。计算机运行程序的时候,会从 `int main()` 后面的大括号开始运行,依次执行大括号中的语句直到结束。举个例子,仍旧是想要烹饪一道菜,那么“炒菜”这个动作就是主函数。炒菜这个主函数里还包含别的步骤,例如“洗菜”“切菜”“下锅翻炒”“调味”“起锅装盘”就是这个主函数里面的语句。

`cout << "I love Luogu !";` 是输出语句,该语句可以让计算机输出一句话(注意是两个英文的小于号而不是中文的左书名号)。有同学会问,既然这一句话就可以达成目的,那为什么还要前面和后面的那么多“废话”呢?的确有的语言(比如 Python)只用一条语句就可以完成任务;但是对于 C++ 来说,这是语言特性所造成的。在之后的学习过程中会自然而然地理解 C++ 拥有这种特性的原因。

`return 0;` 的意思是主函数需要返回一个“0”,这说明程序正常运行结束。在主程序中即使不写这一句话,编译器也会在编译时自动加上这句话。注意,千万不要 `return` 一个其他的非 0 值,这会造成系统认为这个程序异常退出。在算法竞赛中,这会导致被认为运行错误。

如果想借助计算机解决实际问题,就要设计计算机程序并让计算机执行。在这里举一个非常简单的例子来说明使用计算机解决问题的具体步骤。

例 1-2 有 10 个苹果,小 A 拿走了 2 个,Uim 拿走了 4 个,八尾勇拿走了剩下的所有苹果。想知道:

- 1) 小 A 和 Uim 两个人一共拿走了多少苹果?
- 2) 八尾勇能拿走多少苹果?

现在需要编写一个程序,输出两个数字作为答案,中间使用空格分开。可以通过下面的步骤来解决这个问题(如图 1-2 所示)。

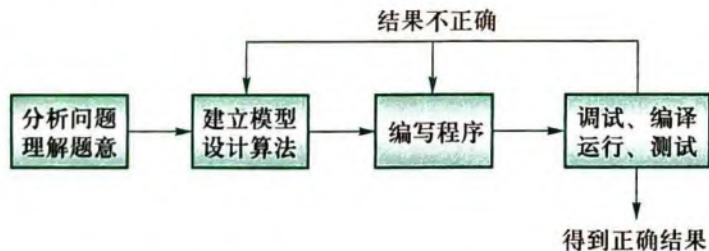


图 1-2 解决算法问题的步骤

(1) 分析问题,理解题意

这是个非常简单的小学应用题。首先是确定程序“做什么”。这个题意非常明确,就是要求出两个人拿到的苹果总数和第三个人拿到的苹果数量。但是,不是所有问题的题意都是这么直

白的,有些问题描述冗长复杂,需要从题目描述中知道能给出什么输入,需要怎么处理数据,以及如何输出这些数据。后面会接触不少需要仔细揣摩题意的题目。

(2) 建立模型,设计算法

接下来需要考虑程序“如何做”,也就是设计一个算法。算法是指一套确定的、有限的、能解决特定问题的流程。本题的算法很简单,就是直接输出答案(2+4)和(10-2-4)。后面将会设计很多较复杂的算法。

(3) 编写程序

下面将使用 C++ 来实现算法。根据题意,可以写出如下参考程序:

```
#include<iostream>
using namespace std;
int main() {
    cout << 2 + 4 << " " << 10 - 2 - 4;
    return 0;
}
```

(4) 调试、编译运行与测试

写完程序后,很可能程序中存在一些错误。有些错误(编译错误)可以被编译器发现并被拒绝运行,比如漏掉了一个分号“;”,而有的错误(逻辑错误)虽然可以运行,但并不会返回期望的结果,例如加减号录入反了。根据编译器的提示,更正编译错误后让计算机编译运行,检查是否是期望的结果。如果出现了偏差,那么就需要检查是否存在逻辑错误。甚至有时候会发现自己理解错了题意或者算法本身是不正确的,如果是这样,就要回到前面几个步骤重新设计。

运行该程序,可以看到计算机的输出:

6 4

和期望的结果一致。这个程序和例 1-1 的程序很像。其中的头文件、命名空间、主函数就不再解释了。计算机从主函数的大括号后开始执行语句。cout 语句分别输出了 2+4、“ ”、10-2-4 这 3 个项目,这说明 cout 语句可以同时输出多个项目,中间用 << 分割。首先计算并输出了第 1 项,即数字“6”;然后输出了第 2 项,即一个空格;接着又计算并输出了第 3 项,数字“4”。和例 1-1 一样,双引号内的内容可以随意更改,比如换成两个空格或者一个逗号等。

到现在为止已经给出两个例子程序了,是不是比想象的简单一些?

陆游说过:纸上得来终觉浅,绝知此事要躬行。强烈建议读者上机编写程序而不是仅仅阅读本书。^①虽然这样可能会花不少时间,但这是学习编程的必要过程。

^① 甚至读者在练习的时候可以尝试“故意犯错”,比如少写分号,使用全角符号,漏写命名空间声明语句等,看看会产生什么后果。自己先踩一遍坑后,遇到相同的问题就可以很快解决了。

1.2 简单数学运算

计算机,顾名思义,就是非常擅长做“计算”的机器。上一节已经介绍了计算加减的方式,现在介绍更多的计算符号。

例 1-3 分苹果。现在有 14 个苹果,要均分给 4 名同学,分不掉的苹果放回冰箱。请问:

- 1) 每位同学能分得几个苹果?
- 2) 一共分出去多少个苹果?
- 3) 把几个苹果放回冰箱?

现在需要编写一个程序,输出 3 个数字作为答案,每行一个数字。

```
#include<iostream>
using namespace std;
int main() {
    cout << 14 / 4 << endl;
    cout << 14 / 4 * 4 << endl;
    cout << 14 - 14 / 4 * 4 << endl;
    // cout << 14 % 4 << endl;
    return 0;
}
```

运行程序,输出:

```
3
12
2
```

可以发现,符号 / 在这里是整除的意思。在数学中, $14 \div 4 \times 4$ 应该还是 14,但是在 C++ 中, $14/4$ 就是 14 除以 4 后向下取整(也就是去除小数部分),是一个整数,所以会输出 3。第二行就输出了 3×4 ,也就是 12。endl 的意思是输出一个换行。当然,也可以将这 3 行输出语句浓缩成一行,但是这样写出来的程序可读性就比较差了。程序不仅是写给计算机看的,常常人也要看。可读性好的程序可以帮助阅读程序的人更好地理解程序内容。

* 是乘法运算符。它和在小学学到的一样,C++ 中的数学运算也是先乘除再加减。所以,表达式 $14-14/4*4$ 会先计算乘除的部分,得到 12 这个结果后再去和 14 进行减法运算,得到答案 2。

求放回冰箱的苹果数量还有另外一种方法,就是程序中的 $14\%4$,符号 % 的意思就是取余数,也称为求模。取余数的优先级和乘除是一样的。

注意到程序第 7 行的前面是两个斜杠 //,这说明这句话是**程序注释**。计算机运行程序的时候会忽略掉注释这一行,也就是说这一行会被跳过。可以在双斜杠后面添加任何内容来进行解释说明或者记录备忘。可以做个实验,把这一行前面的双斜杠去掉再编译运行程序,看看会输出什么?

如果想先计算 $14-14$ 再计算乘除法,该怎么办呢?那就加上括号,变成 $(14-14)/4*4$ 。可以通过添加括号的方式改变计算优先级。即使不需要添加括号来改变运算优先级,有时候还是可以通过添加括号对表达式中的项目分组,让式子更容易让人理解。需要注意的是,括号前后的乘号不能省略。例如 $3(4-2)$ 是不正确的,必须写成 $3*(4-2)$ 。

这里可以发现,括号的优先级最高,乘除的优先级比括号低,而加减的优先级最低。

例 1-4 均分肥宅水。现在有 500 毫升的肥宅快乐水,要均分给 3 位同学,每位同学可以分到多少毫升?请输出一个数字作为答案。

```
#include<iostream>
using namespace std;
int main() {
    cout << 500.0 / 3 << endl;
    return 0;
}
```

运行程序,输出

166.667

本例中,除号还是使用斜杠/,唯一的不同就是 500 变成了 500.0。在计算机中,500 被认为是整数,而 500.0 是浮点数^①,完全是两码事。整数加减乘除模整数还是整数,而如果希望得到一个带有小数的结果,那么需要将运算符两边至少一个数字以浮点数的形式表示,最简单的方法就是在整数后面加上“.0”。

例 1-5 输入以下程序,观察输出。

```
#include<iostream>
using namespace std;
int main() {
    cout << 500.0 / 3 << endl;
    cout << 5000000.0 / 3 << endl;
    cout << 0.000005 / 3 << endl;
    cout << 5e6 / 3 + 5e-6 / 3 - 5e6 / 3 << endl;
    return 0;
}
```

运行程序,输出:

166.667
1.66667e+06

^① 一些教材中会称之为“实数”,但笔者认为并不准确,因为浮点数能够表示的精度和范围是有限的,不能表示全体实数。

```
1.66667e-06
```

```
1.6666e-06
```

本例中,C++ 使用 `cout` 输出浮点数,默认保留不超过 6 位有效数字。如果数字过大或者过小,那么就会使用科学记数法输出,同样保留不超过 6 位有效数字,比如 `1.66667e+06` 就是 1.66667×10^6 的意思。在程序中也可以使用这样的形式来表示一个浮点数,只是要注意整个浮点数内部不能有空格(例如 `1.667 e - 17` 是不可以的,整个浮点数是一个整体,运算优先级相当高),而且如果指数是正的,那么加号可以省略。

最后一行输出了 `1.6666e-06`,可能与预想的不一样,这是因为计算机的存储方式决定了浮点数能够表示的精度是有限的^①。在计算过程中,如果有效数字的位数超过了一定的限度,那么有效数字的最右端就会被舍去;此外,浮点数的指数范围也是有限的。至于一个浮点数能够有多少位有效数字以及指数能到多少,会在后面的章节讨论。

例 1-6 火车问题。甲列火车长 260m,每秒前进 12m;乙列火车长 220m,每秒前进 20m,两车相向而行,从两车车头相遇时开始计时,多长时间后两车车尾相离? 已知答案是整数。

分析: 如图 1-3 所示,以甲车为参考系(就是站在甲车上观察,甲车是静止的),乙相对于甲走了 $220\text{m} + 260\text{m} = 480\text{m}$ 。它们的相对速度是 $12\text{m/s} + 20\text{m/s} = 32\text{m/s}$,所以所需的时间是 $480\text{m} / 32\text{m/s} = 15\text{s}$ 。

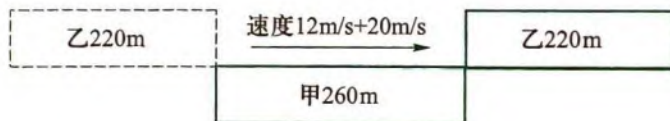


图 1-3 火车的相对路程和速度

```
#include<iostream>
using namespace std;
int main() {
    cout << (260 + 220) / (12 + 20) << endl;
    return 0;
}
```

考虑题目中已经保证答案是整数,所以可以使用整除。但是在很多情况下,不能保证中间结果一定是整数,所以常常需要考虑是否要转换为浮点数计算(也就是加上“.0”)。

例 1-7 对角线问题。一个长方形的长和宽分别是 6cm、9cm,求它的对角线长度。

分析: 根据勾股定理,答案就是 $\sqrt{6^2 + 9^2}$ 。

```
#include<iostream>
#include<cmath>
using namespace std;
```

① 实际上,计算机能够精确表示的浮点数只有 2 的整数次幂(包括负数次幂)和它们的整数倍数,其余的都是近似值。

```
int main() {
    cout << sqrt(pow(6, 2) + pow(9, 2)) << endl;
    return 0;
}
```

这里使用了新的头文件 `cmath`。从这个名字可以看出,这个头文件跟数学有关。这个程序用到了 `pow()`、`sqrt()` 两个函数,它们都是在 `cmath` 头文件中的。

函数可以被认为是一种实现特定功能的工具,把数据调入对应功能的函数中,这个函数就会在计算后返回值。不同函数的输入变量和输出类型会不同,有的函数甚至没有输入或者输出。

`pow(6,2)` 的意思是计算 6 的 2 次方,返回的是一个浮点数。这里也可以使用 `6*6` 达到同样的效果。

`sqrt()` 的意思是计算括号里的算术平方根,返回的也是一个浮点数。

最后把这个数字输出。

常用 `cmath` 头文件整理:头文件 `cmath` 还包括很多其他的数学函数,见表 1-1。这张表的“函数原型”告诉用户如何去调用这个函数,以及这个函数的返回值是什么类型。例如 `double pow(double x, double y)` 的意思是 `pow` 函数需要输入两个 `double` 类型的数字 `x` 和 `y`,输出一个 `double` 类型的数字。如果输入的 `x` 或 `y` 是 `int`、`float` 等类型,就会自动转换为 `double` 类型。

表 1-1 数 学 函 数

函数原型	样例	说明
<code>double sin(double x)</code> <code>double cos(double x)</code>	<code>sin(3.14159/2)</code>	三角函数正弦和余弦, x 是弧度
<code>double exp(double x)</code>	<code>exp(1)</code>	返回 e^x , 其中 e 是自然常数
<code>double log(double x)</code>	<code>log(10)</code>	返回 x 的自然对数
<code>double pow(double x, double y)</code>	<code>pow(3, 2)</code>	返回 x^y , 也可以用来运算多次根式
<code>double sqrt(double x)</code>	<code>sqrt(9)</code>	返回 $\sqrt{x}$
<code>double fabs(double x)</code>	<code>fabs(-10)</code>	返回 x 的绝对值
<code>double ceil(double x)</code>	<code>ceil(2.1)</code>	返回大于或等于 x 的最小的整数值(上取整)
<code>double floor(double x)</code>	<code>floor(2.9)</code>	返回小于或等于 x 的最大的整数值(下取整)

当然现在不理解 `double` 之类的也没关系,在之后的章节中将会讲到。

1.3 变量与常量

学到这里,有些同学可能会认为 C++ 也不过就是一个计算器罢了。但是 C++ 能够完成的任务远不止计算几个表达式这么简单。有的问题是需要通过多个表达式计算才能解决的,因此需要有一个办法能够储存计算的中间结果。这时,可以使用**变量**来满足这个要求。

例 1-8 存钱。Uim 的银行账户里面有 100 元。经过了下面的操作:

- 1) 往里面存了 10 元;
- 2) 购物花掉了 20 元;

3) 把里面的钱全部取出。

请在每次操作后输出账户余额,并使用换行符隔开。

分析:可以写出如下程序。

```
#include<iostream>
using namespace std;
int main() {
    int balance = 100; // 初始余额
    balance = balance + 10;
    cout << balance << endl;
    balance -= 20;
    cout << balance << endl;
    balance = 0;
    cout << balance << endl;
    return 0;
}
```

运行程序,输出:

```
110
90
0
```

程序定义了 balance 这个 int 类型的变量,同时赋上初始值 100。int 是一种最基本的整型变量,可以储存一个整数。这相当于在厨房操作台上面摆上一个专门用来放整数的碗,这个碗的名字是 balance,里面装有 100 这个整数。后面的注释可以省略!

接下来, balance = balance + 10 是**赋值语句**。通俗来讲,赋值就是在碗中放东西。虽然有一个 = 符号,但这并不是说明等号的左边和右边是相等的。这条语句的意思是先计算表达式 balance + 10 的值,然后把这个计算结果放到 balance 中(原来的 balance 数据会被丢弃)。

balance -= 20 也是**赋值语句**,但确切地说是**自赋值**。这条语句的意思是把 balance 这个变量取出来,然后减去 20,最后再放回去,其效果和 balance=balance-20 是一样的。除了 -=,还能使用 +=、*=、/= 和 %= 来进行自赋值。读者可以尝试改变程序的第 5 行,使用自赋值语句。

balance = 0 说明了还可以给变量赋值一个数字,赋值后变量就变成了等号右边的值。

例 1-9 当半径为 $r=5$ 时,请输出圆的周长、面积和球体积。取 $\pi=3.141593$ 。

分析:根据公式,圆的周长 $C=2\pi r$,圆面积 $S=\pi r^2$,球体积 $V=\frac{4}{3}\pi r^3$ 。可以写出如下程序:

```
#include<iostream>
#include<cmath>
using namespace std;
int main() {
    double r = 5;
```

```

    const double PI = 3.141593;
    // #define PI 3.141593
    cout << 2 * PI * r << endl;
    cout << PI * r * r << endl;
    cout << 4.0 / 3 * PI * pow(r, 3) << endl; // 不能写成 4/3*PI*pow(r,3)
    return 0;
}

```

程序多次用到表示圆周率的PI,因此可以将其定义为常量。只需要像定义变量那样,在数据类型的前面或者后面加上const,就可以定义常量。常量一经定义就不能在程序运行过程中修改了(否则会出现编译错误)。像这样固定不变的数字就非常适合定义为常量,习惯上将常量名用大写字母定义。

还有另外一种方法。如果使用#define宏定义的方法,程序编译时会将这行后面所有代码中的PI简单粗暴地替换成3.141593,基于这样的特性常常会用这种方法来定义常量,而不需要确定其数据类型。

变量和常量的命名不是想怎么命名就怎么命名的,需要遵循以下要求:

1) 只能由英文字母、数字和下画线(_)组成。

2) 不能以数字开头。

3) 不能和其他“关键字”重复。关键字(又称保留字)有很多,如int、if等。并不需要知道哪些是关键字,如果出现编译错误,换一个词语即可。

还需要注意的是,变量名区分大小写,比如Ans和aNS是两个不同的变量;而且变量命名应当一看就知道是什么意思,如果全部的变量都是a、b、x、y之类的没有明确意思的字符,可能过一段时间自己都不知道每个变量是什么意思了,造成代码理解混乱。

 **例 1-10 猴子吃桃。**一只小猴买了若干桃子。第一天它吃了这些桃子的一半之后,又贪嘴多吃了一个;第二天它也吃了剩余桃子的一半,又贪嘴多吃了一个;第三天它又吃了剩下的桃子的一半,并贪嘴多吃了一个。第四天起来一看,发现桃子只剩下一个了。请问:小猴买了几个桃子?

分析:直接求解本题比较难。但是,如果从最后一天倒推回去,那就简单多了。第四天剩下一个;第三天吐出一个,然后翻倍(因为反过来是吃掉一半后又吃掉一个)。第二天和第一天也是同样的操作。因此可以写出如下程序:

```

#include<iostream>
using namespace std;
int main() {
    int num = 1; // 第四天
    num = (num + 1) * 2; // 第三天
    num = (num + 1) * 2; // 第二天
    num = (num + 1) * 2; // 第一天
    cout << num << endl;
    return 0;
}

```

这里定义了一个整型变量 `num` 来记录每一天的桃子的数量。这里的赋值语句不是说左右两边是相等的方程,而是将右边计算出来的结果赋值进左边的变量,同样的操作需要重复 3 次。

虽然本例完全可以写成一行 $((1+1)*2+1)*2+1$,这个结果也是对的,但是式子比较复杂,不容易一下子就能看清楚中间的过程,而这种写法就清晰多了。

例 1-11 (选读)评测机队列。洛谷的评测任务是单位时间内均匀增加的。8 台评测机 30min 可以刚好把评测队列中的程序评测完毕,10 台评测机 6min 可以刚好把评测队列中的程序评测完毕。请问:几台评测机可以在 10min 时刚好把评测队列中的程序评测完毕?

分析:这是著名的“牛吃草问题”的模型。假设 1 台评测机 1min 可以评测 1 份程序。

首先需要分析每分钟有多少新程序进入评测队列。8 台评测机 30min 可以评测 240 份,而 10 台评测机 6min 可以评测 60 份,可以得到 $30\text{min}-6\text{min}=24\text{min}$ 内,增长了 $240\text{份}-60\text{份}=180$ 份程序。因此,每分钟程序的增长速度是 $(240\text{份}-60\text{份})/(30\text{min}-6\text{min})=7.5\text{份}/\text{min}$ 。

6min 可以评测 60 份,其中 $6\text{min}\times 7.5\text{份}/\text{min}=45$ 份是新提交的程序,原有队列里面有 $60\text{份}-45\text{份}=15$ 份程序。评测机 10min 一共需要评测 $15\text{份}+10\text{min}\times 7.5\text{份}/\text{min}=90$ 份程序,所以需要 $90/10=9$ 台。根据这个思路,可以写出如下程序:

```
#include<iostream>
using namespace std;
int main() {
    int n1 = 8, t1 = 30, n2 = 10, t2 = 6;
    // 题目给出的评测机数量和时间
    int t3 = 10; // 题目要求的时间
    double inc_rate = (1.0 * n1 * t1 - n2 * t2) / (t1 - t2); // 增长速度
    double init_num = n2 * t2 - inc_rate * t2; // 初始队列长度
    double ans = (init_num + t3 * inc_rate) / t3; // 求得答案
    cout << ans;
    return 0;
}
```

这个问题相比于前面的问题来说比较复杂,很难使用一个表达式来直接求得结果。可以把一个大任务拆分成若干规模比较小的任务,抽丝剥茧,逐步击破,直到求得最终的答案。

编写程序时应注意对变量的命名。变量不需要起很长的名字,但是至少要一看到变量名就知道它是干什么的。如果这种有明确意义的变量使用 `a`、`b`、`c` 之类的变量名,那么容易让人觉得困惑。一份可读性良好的程序不仅方便供他人观看,也能够使自己编写程序时思路更加清晰。

1.4 课后习题与实验

习题 1-1 请将下列公式翻译成表达式。

(1) $3x+5y$

(2) $\frac{c+1}{ab}$

- (3) $\sqrt{3a^3}$
- (4) $(n+2)(n-9)$

习题 1-2 下列变量名中,哪些是合法(可通过编译)的,哪些是非法的?

- (1) kkksc03
- (2) OhILoveLuoguVeryMuchAndIWillStudy
- (3) _lapple
- (4) char
- (5) kkk@SH
- (6) a
- (7) iPhone
- (8) 11dimensions
- (9) __stdcall(两个下画线)

习题 1-3 安装配置好编程环境,将本章例题的所有代码输入到计算机中,亲自运行这些程序。

习题 1-4 编写程序解决以下问题,然后手工计算,验证答案。

- (1) 3 名同学 3h 可以扫干净 3 间教室,那么 9 名同学 9h 可以扫干净几间教室?
- (2) 长方形的长和宽之和是 24cm,长比宽多 4cm。请问:长方形的面积是多少?
- (3) 小 A 和 Uim 在程序设计竞赛中的得分之和是 480,Uim 的得分是小 A 的 1.4 倍。请问:他们分别得了多少分?
- (4) 给同学分苹果,若每人分 3 个就剩下 11 个;如果每人分 4 个则少一个。请问:有多少位同学,有多少个苹果?
- (5) 小 A 每分钟输入 120 个字符,Uim 每分钟输入 80 个字符,Uim 比小 A 先开始打字 12min。请问:小 A 开始打字多少时间后能赶上 Uim 的进度?
- (6) 兔子有 4 只脚,鸡有 2 只脚。一个笼子里面有若干只兔子和鸡,有 35 个头,94 只脚。请问:兔子和鸡分别有几只?
- (7) 银行定期存款年利率是一年定存 3.5%,五年定存 4%。小 A 和 Uim 手上各有 10000 元。小 A 决定每次存一年期,到期后将连本带利再存一年,直到存满 5 年。Uim 直接存五年定期。请问:5 年后他们分别有多少钱?

第2章 顺序结构程序设计

在编写计算机程序时,将一个任务分解成一条一条的语句,计算机将按照顺序一条一条地执行这些语句,这就是顺序结构程序设计。

接着上一章的内容,本章引入变量数据类型的概念,介绍几种常见的、用于不同场合的变量类型。此外,用户希望将数据输入计算机,计算机就能给输出数据作为答案,帮助用户完成一个具体的编程任务,与人类交互。如果读者在第1章已经顺利解决了这个问题,那么对本章内容应该不会觉得太难。图2-1所示为本章思维导图。

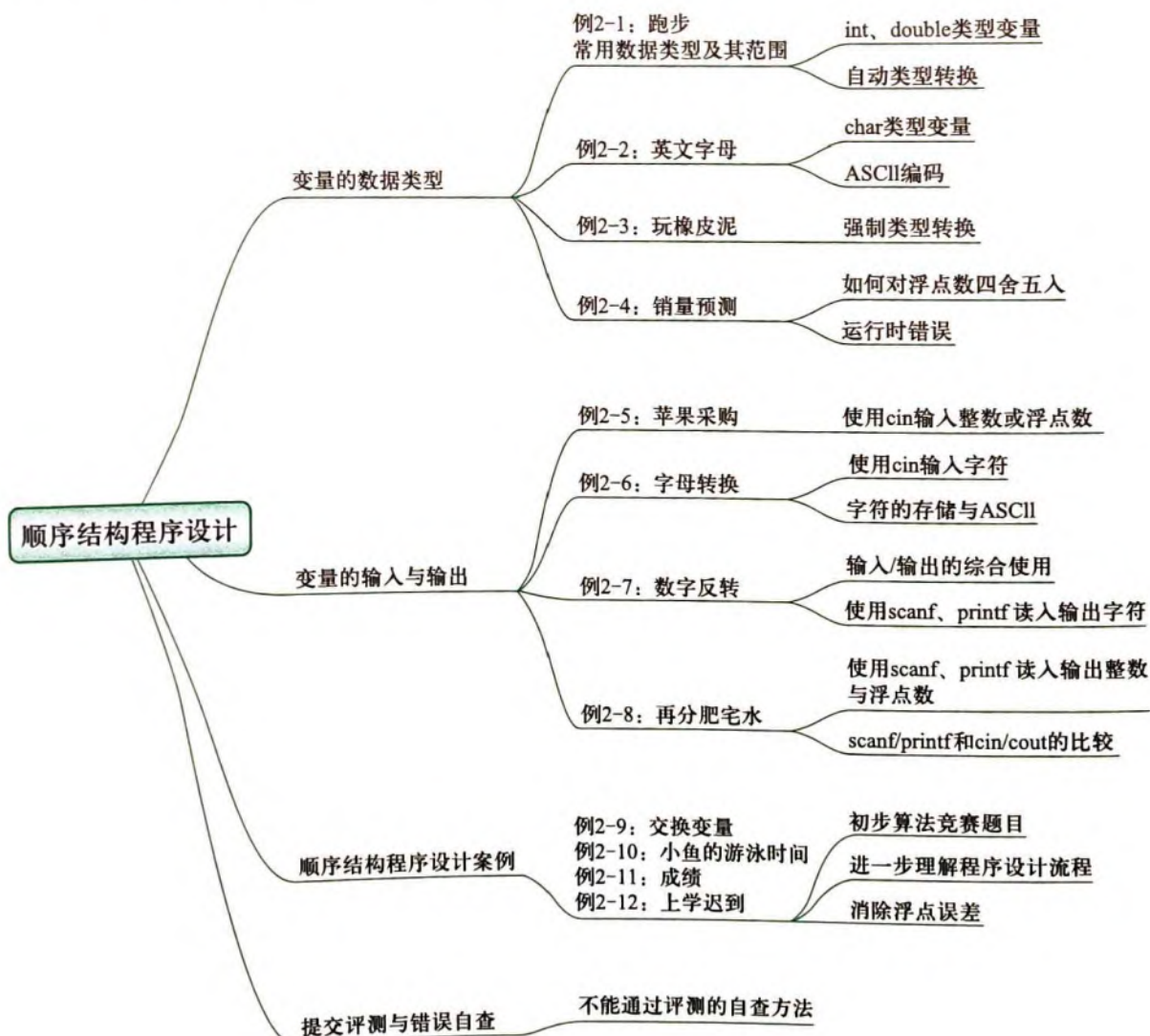


图 2-1 本章思维导图

2.1 变量的数据类型

还是以做菜为例子。首先处理好了一种原材料,接着要去处理另外一种。不可能将所有的原材料都拿在手上,因此需要找一个容器,把原料放在容器里,并放在操作台的空位,等需要的时候再拿来用。在这里,原材料相当于要处理的数据,容器就是变量,而操作台就是内存。一般会把一只鸡和一撮葱花装在同样大小的容器中,也不会把熬好的高汤倒入菜篮子。C++ 语言有很多种变量类型,能够存储的数据精度和范围都不一样,所以要根据数据的实际情况选择合适的变量类型。

例 2-1 跑步。小 A 的跑步速度为 5m/s,八尾勇跑步的速度为 8m/s,八尾勇在小 A 后面 100m,他们同时起跑。请问:需要多长时间八尾勇可以追上小 A? 输出一个数字表示答案。

分析:八尾勇和小 A 之间的相对速度是 $(8-5)\text{m/s}=3\text{m/s}$ 。也就是说,每秒钟他们的相对距离就会缩短 3m,因此 $100/3$ 就是答案。

```
#include<iostream>
using namespace std;
int main() {
    int v_a = 5, v_yao = 8, distance = 100; // 小 A 和八尾勇的速度, 以及距离
    double delta, ans; // 速度的差值和答案
    delta = v_yao - v_a; // 两人的相对速度, 也就是每秒距离缩短多远
    ans = distance / delta;
    // ans = 1.0 * distance / (v_yao - v_a)
    cout << ans << endl;
    return 0;
}
```

运行程序,输出:

33.3333

最开始定义了 `v_a`、`v_yao`、`distance` 这 3 个 `int` 类型的变量并赋初始值。然后定义了 `delta`、`ans` 这两个 `double` 类型的变量,但是没有给它们赋初始值。`double` 类型的意思是双精度浮点数,暂时可以认为是能存放带小数的数字。

需要注意的是,如果一个在函数内(当然包括主函数 `main`)的变量没有被初始化,就不能拿来参与计算(但是可以被赋值)。设想一下,指定了一个变量碗,但是这个碗是之前程序用过的然后被废弃,里面不知道有什么奇怪的东西,如果直接拿来用搞不好就会食物中毒。因此,在使用这个变量前,必须给这个变量赋值,进行初始化。

正常情况下,赋值符号的右边的数据类型(整数或者是浮点数)应该和左边的变量数据类型相匹配,但是有些情况也可以不一致。正如程序第 6 行,赋值右边是两个整数的差,结果还是整数;但是左边的变量类型是浮点数,所以右边的整数类型会被自动转换成左边的浮点数的类型。相反,如果右边是浮点数类型,左边是整数类型,C++ 在自动转换的时候会舍去浮点数的小数部

分,这说明跨类型的自动转换可能会造成一些精度损失。

计算答案 `ans` 的时候, `distance` 是整数类型, `delta` 也是浮点数类型,所以它们相除会变为浮点数类型,刚好可以存在 `ans` 变量里。这种现象被称为**自动类型转换**。

还有另外一种写法可以不使用 `delta` 变量,就像注释行那样。因为右边的变量全部都是整数,所以需要乘上 1.0 将 `distance` 变成一个浮点数;否则,计算机就将其当作整除来处理造成答案错误。

常见的数据类型^①见表 2-1。

表 2-1 常见数据类型

数据类型	占用空间	取值范围
<code>char</code>	1 字节, 8 位	-128~127
<code>int</code>	4 字节, 32 位	$-2^{31} \sim 2^{31}-1$, 大约能够表示绝对值不超过 2.1×10^9 的整数
<code>unsigned int</code>	4 字节, 32 位	$0 \sim 2^{32}-1$, 大约能够表示不超过 4.2×10^9 的非负整数
<code>long long</code>	8 字节, 64 位	$-2^{63} \sim 2^{63}-1$, 大约能够表示绝对值不超过 9.2×10^{18} 的整数
<code>unsigned long long</code>	8 字节, 64 位	$0 \sim 2^{64}-1$, 大约能够表示不超过 1.8×10^{19} 的非负整数
<code>float</code>	4 字节, 32 位	大约指数绝对值不超过 37, 6 位有效数字
<code>double</code>	8 字节, 64 位	大约指数绝对值不超过 307, 15 位有效数字

取值范围中的“大约”是保守的估计,有时候即使稍微超过也能表示(也有可能不能),但是不超过这里的范围是可以确保准确的。另外可以看出,这些数据类型默认是带符号的(也就是 signed),可以表示正数、负数或 0。如果在数据类型前面加上 `unsigned`,就会变成无符号数:只能表示 0 或者正数,不过能够表示的正数范围相对于带符号数扩大了一倍。

从 2-1 表可以看出, `long long` 可以表示更大范围的数字。那么,能不能只使用这些大范围的数据类型而不用其他的类型呢? 答案是否定的。一方面,计算机内存大小是有限的,如果选用这些数据类型,则能够容纳的数字位数就变少了,毕竟这些数据类型占用空间比较大;另一方面,对于一般的运算来说,大范围的数据类型运算速度比较慢(做一个粗浅的类比:做随机 5 位数乘 5 位数的运算速度肯定会比随机 10 位数乘 10 位数要快)。

因此,权衡精度和空间与速度,一般情况下使用 `int` 数据类型来存放整数。只有当 `int` 无法存下的一些特别大的数字时,才会去使用 `long long`;而浮点数如果因为空间有限制时选择 `float`,否则选择 `double`。

例 2-2 英文字母。英文有 26 个字母,其中 A 是第一个字母。现在请编程求出:

1) M 是字母表中的第几个字母?

2) 第 18 个字母是什么?

输出两个数字,使用换行隔开。

分析:首先给出如下代码。

① 不同编译器下的取值范围可能与表 2-1 中不同,本书统一按照 GCC8.2 32 位处理器的标准,多数算法竞赛的编译器和这个标准一致。